

Laboratorio I

a.a. 2025/2026

Esercitazione argomenti primo semestre

Esercizio 1 - FilteredSet

Si scriva una funzione JavaScript `filteredSet(S, p)` che, dato un multi-insieme `S` codificato come visto a lezione e un predicato `p` da stringhe a booleani, restituisca un nuovo multi-insieme contenente solo gli elementi di `S` che soddisfano `p`, senza modificare `S`.

La funzione deve restituire il nuovo multi-insieme, e il numero di elementi cancellati.

Esercizio 2 - ksumlimit

Si scriva una funzione JavaScript

`ksumlimit(p, k, n1, n2, n3, ...)`.

La funzione restituisce un array con i primi numeri,

incontrati scorrendo da sinistra verso destra,

strettamente maggiori di `p`

la cui somma totale non supera `k`.

ESEMPI:

`ksumlimit(3, 20, 5, 10, 4, 8, 1, 7) --> [5, 10, 4]`

Esercizio 3 - Prodotto estremi

Si scriva una funzione in JavaScript `prodEstremi(A)` che, dato un array `A` con elementi numerici, restituisca un nuovo array contenente in posizione 0 il prodotto del minimo e del massimo di `A`; in posizione 1 il prodotto del secondo più alto e del secondo più basso elemento di `A`, ecc. Se `A` è di lunghezza dispari, il suo elemento mediano viene moltiplicato per se stesso (e il risultato sarà l'ultimo elemento dell'array risultante). L'array `A` non deve essere modificato.

Esempi:

`[2,8,4,1,0] --> [0, 4, 4]`

`[10,3,7,1,0,3,1,4] --> [0, 7, 4, 9]`

Esercizio 4a - Costruttore di pile

Si scriva una funzione JavaScript `nuovaPila()` con le seguenti caratteristiche:

- la funzione deve restituire un nuovo oggetto (chiamiamolo `P`), avente (almeno) due chiavi `impila` e `depila`
- il valore di `impila` deve essere una funzione che, dato un valore, lo inserisce in cima alla pila (memorizzandolo in un modo a vostra discrezione in `P`)
- il valore di `depila` deve essere una funzione che, senza prendere nessun argomento, rimuove e restituisce l'elemento in cima alla pila (memorizzato in precedenza da `impila`). Se la pila è vuota, deve restituire `undefined`.

Attenzione: si noti che ogni pila restituita da una chiamata a `nuovaPila()` è distinta da ogni pila restituita da una diversa chiamata a `nuovaPila()`

Esercizio 4b - Costruttore di coda

Si scriva una funzione JavaScript `nuovaCoda()` con le seguenti caratteristiche:

- la funzione deve restituire un nuovo oggetto (chiamiamolo `Q`), avente (almeno) due chiavi `enqueue` e `dequeue`.
- il valore di `enqueue` deve essere una funzione che, dato un valore qualunque, lo inserisce in coda (memorizzandolo in un modo a vostra discrezione in `Q`)
- il valore di `dequeue` deve essere una funzione che, senza prendere nessun argomento, rimuove e restituisce il primo elemento dalla coda (memorizzato in precedenza da `enqueue`). Se la coda è vuota, questa funzione deve restituire `undefined`.

Attenzione: si noti che ogni coda restituita da una chiamata a `nuovaCoda()` è distinta da ogni coda restituita da una diversa chiamata a `nuovaCoda()`.

Esercizio 5 - Il buon giardiniere

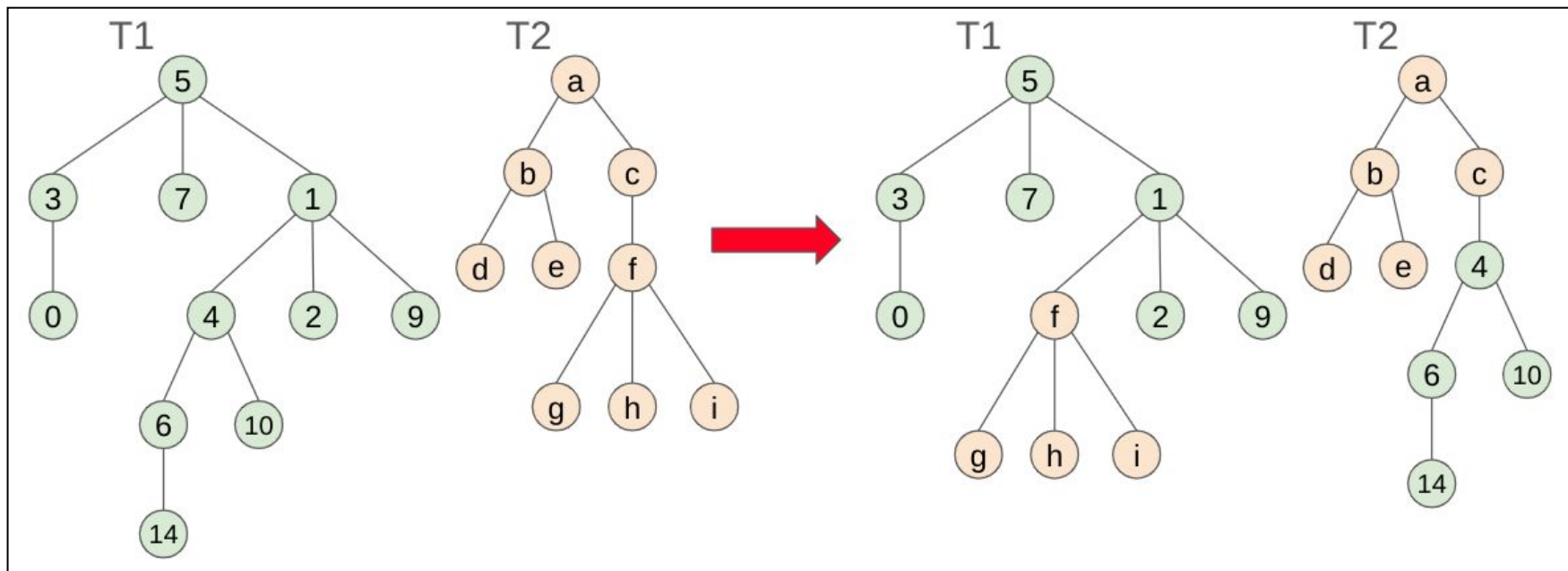
Si scriva una funzione JavaScript `innesta(T1,v1,T2,v2)` con il seguente funzionamento:

- `T1` e `T2` sono alberi k -ari, realizzati come visto a lezione; è garantito che i valori dei nodi di `T1` e di `T2` siano tutti distinti
- `v1` e `v2` sono due valori; è garantito che `v1` compaia in `T1`, e che `v2` compaia in `T2`; è garantito anche che `v1` e `v2` non saranno i valori dei nodi radice di `T1` e `T2`
- la funzione deve scambiare il sottoalbero di `T1` radicato nel nodo con valore `v1`, con il sottoalbero di `T2` radicato nel nodo con valore `v2`

Esercizio 5 - Il buon giardiniere (cont)

Esempio

- La figura mostra T1 e T2 prima e dopo la chiamata a `innesta(T1,4,T2,"f")`



SOLUZIONI

Esercizio 1 - filteredSet

```
function filteredSet(S,p){  
    let copyS = {...S}  
    let cont=0  
    for (let chiave in copyS){  
        if (!p(chiave)){  
            cont += copyS[chiave]  
            delete copyS[chiave]  
        }  
    }  
    return [copyS, cont]  
}
```

Esercizio 2 - ksumlimit

```
function ksumlimit(p, k, ...n){  
  let nfilt = n.filter((x)=>(x>p))  
  let sum = 0  
  for (let i = 0; i < nfilt.length; i++){  
    sum += nfilt[i]  
    if (sum > k) return nfilt.slice(0,i)  
  }  
  return nfilt  
}
```

Esercizio 3 - Prodotto estremi

```
function prodEstremi(A){  
    let copyA = [...A]  
    copyA.sort((a,b)=>a-b)  
    let newA = []  
    let half = Math.ceil(copyA.length/2)  
    for (let i=0; i<half; i++){  
        newA.push(copyA[i]*copyA[copyA.length-i-1])  
    }  
    return newA  
}
```

Esercizio 4a - Costruttore di pile

```
function nuovaPila(){  
  let A=[]  
  return {impila: (v)=>A.push(v) , depila: ()=>A.pop() , pila:A }  
}
```

Esercizio 4b - Costruttore di code

```
function nuovaCoda(){  
  let A=[]  
  return {enqueue: (v)=>A.push(v) , dequeue: ()=>A.shift() , coda:A }  
}
```

Esercizio 5 - Il buon giardiniere

```
function innesta(T1,v1,T2,v2){  
    let [A1,i] = trovav(T1,v1)  
    let [A2,j] = trovav(T2,v2)  
    let tmp = A1.figli[i]  
    A1.figli[i] = A2.figli[j]  
    A2.figli[j] = tmp  
}
```

```
function trovav(T,v){  
    if (T.figli==undefined) return [undefined, null]  
    for (let i=0; i<T.figli.length; i++){  
        if (v==T.figli[i].val) return [T,i]  
        let [A,k] = trovav(T.figli[i], v)  
        if (A!=undefined) return [A, k]  
    }  
    return [undefined, null]  
}
```